

Active Learning Simulation

1. Generating Dataset

Before running, prepare special dataset from raw data as follows:

```
dataset/
├── images/
│   ├── apracom_bc_84.png
│   ├── apracom_bc_1.png
│   ├── apracom_etb_41.png
│   ├── apracom_ym_150.jpeg
│   └── ... (all 80 images)
└── annotations/
    ├── apracom_bc_84.json # contains bounding boxes and class labels per image
    └── ...
```

Each annotation file should contain, for one image, a list of:

```
{
  "bbox": [cx, cy, w, h], # normalized center coordinates and size (0..1)
  "class": "BC" | "ETB" | "YM" | "XSA" # exactly one of the four
}
```

2. Active Learning Logic

- Loads your annotated dataset (images + bounding boxes + class labels).
It expects a folder structure where each image has a corresponding .json annotation file.

The script includes a placeholder for you to fill in your specific parsing logic.

- Initial seed: 20 images (5 per class) from the 64-image AL pool.
Holdout: 16 test images (4 per class), never used for training or querying.
- Uses a frozen DINOv3 ViT-S/14 backbone or ResNet50(for demo only) to extract features from image crops of each detected colony.

Because your real AdaptiveDETR uses DINOv3 internally, this mimics the same representation power.

- Trains lightweight classification heads (one per bacteria class: BC, ETB, YM, XSA) on top of the frozen DINOv3 features.
- Simulates active learning rounds:
Round 0: Randomly select a small seed set (e.g., 20 images, 5 per class). Train the heads.

For each subsequent round:

- Run inference on all remaining unlabeled images.
- For each image, compute the average entropy of per-colony predictions (uncertainty).
- Select the top batch_size most uncertain images.
- Simulate human annotation – because we already have the ground truth labels, we “add” those images to the training set (in reality, a human would label them).
- Retrain the heads on the enlarged labeled set.

Repeat for, say, 5 rounds.

- Plot / CSV

X: number of labeled images; Y: test accuracy.

Outputs: active_learning_results.png, active_learning_simulation.csv
(num_labeled_images, accuracy).

3. How to use it

Place all 80 images in dataset/images/

Create annotation JSON files (one per image) in dataset/annotations/ with the structure described above.

Update IMAGE_DIR and ANNOT_DIR in the script.

Run the simulation: python active_learning_demo.py

Console logs showing accuracy per round.

4. Summary

The simulation shows, using your own 80 images, how many labeled images would be needed to reach a target accuracy if you use uncertainty sampling. In most cases, active learning requires 40–60% fewer annotations to achieve the same performance.

Because your real AdaptiveDETR uses the same DINOv3 backbone, the relative gain will be very similar. After the demo, I can integrate the uncertainty ranking module directly into your production pipeline as part of the full project.